Ders 12 Genişletilmiş Notu: Sık Öğe Kümeleri ve Birliktelik Kuralları: Apriori, PCY, Multistage ve Multihash

Ders 12 Genişletilmiş Notu: Sık Öğe Kümeleri ve Birliktelik Kuralları: Apriori, PCY, Multistage ve Multihash

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Giriş: Kitabın 6. Bölümü
- 2. Pazar Sepeti Modeli
- 3. Destek Eşiği ve Sık Öğe Kümeleri
- 4. Uygulamalar ve Ölçek Sorunu
- 5. Birliktelik Kuralları ve Güven
- 6. Sık Kümelerden Kural Çıkarma Algoritması
- 7. Verinin Saklanışı ve Maliyet Ölçüsü
- 8. Saf (Naive) Algoritma
- 9. Çift Sayaçlarını Bellekte Tutmak: Üçgen Matris ve Üçlü Tablo
- 10. Apriori Algoritması
- 11. PCY Algoritması
- 12. Multistage Algoritması
- 13. Multihash Algoritması
- 14. Algoritmaların Karşılaştırması
- Örnek Sorular
- Dersin Toparlaması

1. Giriş: Kitabın 6. Bölümü

Bu derste Mining of Massive Datasets kitabının 6. bölümüne, sık öğe kümelerine (frequent itemsets) geçildi. Hoca, bu bölümün slaytlarının da diğerleri gibi ders sitesinde olduğunu söyledi.

2. Pazar Sepeti Modeli

Sık öğe kümesi problemi pazar sepeti modeli (market basket model) üzerinden anlatılır. Modelde iki temel kavram vardır:

- **Öğeler (items):** Büyük bir öğe kümesi. Örneğin bir süpermarkette satılan ürünler.
- **Sepetler (baskets):** Büyük bir sepet kümesi. Alışveriş yaparken ürünleri koyduğumuz sepetler. Her sepet, öğe kümesinin küçük bir alt kümesidir. Hoca sepete işlem (transaction) de dendiğini söyledi.

Amaç, aynı sepette sık birlikte bulunan öğeleri bulmaktır. Hocanın örneği ekmek ve süttü: Bu iki ürün çoğu zaman birlikte satılıyorsa, bu satış ya da pazarlama bölümü için önemli bir bilgidir. Bu yüzden veri kümesindeki bütün sepetlere (işlemlere) bakılır ve çoğu zaman aynı sepette bulunan öğeler aranır.

3. Destek Eşiği ve Sık Öğe Kümeleri

Bir öğe kümesinin desteği (support), o kümedeki öğelerin veri kümesinde birlikte geçtiği sepet sayısıdır. Destek eşiği (support threshold) s, bir öğe kümesinin sık olup olmadığına karar verir: Destek en az s ise küme sıktır.

3.1 Oyuncak Örnek

Hoca slaytta küçük bir örnek gösterdi. Veri kümesinde 5 farklı öğe (m, c, b, j, p) ve 8 sepet var. Soruda destek değeri 3 olarak verilmiş. Yani bir öğe kümesi en az 3 farklı sepette geçiyorsa ona sık diyeceğiz.

Hoca önce tekli öğe kümelerine baktı:

- m, beş farklı sepette geçiyor. Bu 3'ün üstünde, sık.
- c için hoca "1, 2, 3, 4" diye saydı. Sık.
- b ve j de 3'ün üstünde, sık.

Sonra ikili kümelere geçti:

- $\{m, b\}$: 4 sepette geçiyor. Sık.
- $\{b, c\}$: 4 sepette geçiyor. Sık.
- Üçüncü bir çift de sık. Altyazıda bu çift "J and J" diye geçiyor.

  Ders sonrası not: Hoca sayıları "3'ün üstünde" diye değerlendirdi. Tanımı ise "en az 3 sepette" diye vermişti. Doğrusu "en az s" ($\geq s$) biçimidir; desteği tam 3 olan küme de sıktır.

Hoca, sık kümelerin tekli, ikili, üçlü vb. olabileceğini vurguladı. Aradığımız şey, desteği eşiğin üstünde olan bütün öğe kümeleridir.

> **Ek bilgi (derste anlatılmadı):** Bu oyuncak örnek büyük olasılıkla kitabın ve ders slaytlarının standart örneğidir. Orada sepetler şöyledir: $B_1 = \{m, c, b\}$, $B_2 = \{m, p, j\}$, $B_3 = \{m, b\}$, $B_4 = \{c, j\}$, $B_5 = \{m, p, b\}$, $B_6 = \{m, c, b, j\}$, $B_7 = \{c, b, j\}$, $B_8 = \{b, c\}$. Bu sepetlerle sayılar m:5, c:5, b:6, j:4, p:2 çıkar; sık çiftler $\{m, b\}$:4, $\{b, c\}$:4 ve $\{c, j\}$:3'tür. Bu değerler hocanın söylediği "m beş sepette", "$\{m, b\}$ ve $\{b, c\}$ dört sepette" bilgileriyle uyumludur.

4. Uygulamalar ve Ölçek Sorunu

4.1 Süpermarket

Problemin kökeni süpermarketlerdir. Bu yüzden adı pazar sepeti veri analizi problemidir. Burada öğeler süpermarkette satılan ürünler, sepetler ise ödeme yapmak için kasaya getirdiğiniz ürünlerin kümesidir.

Birlikte alınan sık öğeler bulunursa, bu pazarlama için değerlidir. Hoca iki strateji anlattı:

- **Ürünleri ayırmak:** Birlikte satıldığı bilinen iki ürün mağazanın iki ayrı ucuna konur. Müşteri ikisini birden almak için mağazayı dolaşmak zorunda kalır. Bu, mağazaya daha çok ürün satma şansı verir.
- **Birlikte indirim:** İki ürüne birlikte indirim yapılır ve ikisi aynı yere konup "bu iki üründe indirim var" denir.

Hoca bunun süpermarketlerde iyi bilinen ve çok çalışılmış bir problem olduğunu söyledi.

4.2 İntihal Tespiti

Büyük bir belge kümesinde birbirine benzeyen belgeleri aramak da aynı problemdir. Bu durumda model şöyle kurulur:

- Öğeler = belgeler (çünkü birbirine benzeyen belgeleri arıyoruz).
- Sepetler = cümleler.

Süpermarket örneğine benzer şekilde, bir cümleyi sepet olarak düşünürsek, aynı cümleyi içeren belgeler aynı sepette olur. Böyle cümleler arttıkça bu belgeler sık birlikte geçen belgeler olur, yani birbirine benzerler. Aynı yöntem uygulanınca birlikte çok sık geçen belgeler bulunur. Bu belgeler pek çok ortak cümle içerdiği için intihal adayı sayılabilir.

4.3 Birlikte Geçen Kelimeler

Aynı veriye farklı yaklaşılabilir. Bu kez soru şudur: Belgelerin çoğunda birlikte geçen kelimeler hangileridir? Bu durumda model tersine döner:

- Sepetler = belgeler.
- Öğeler = kelimeler.

Hoca iki modellemeyi karşılaştırdı: İntihalde öğeler belgeler, sepetler cümlelerdi ve birbirine benzeyen belgeler aranıyordu. Burada ise çok sayıda belgede sık geçen kelimeler aranıyor.

4.4 Neden Büyük Veri Problemi?

Öğe ve sepet sayısı tek bir belleğe ya da tek bir bilgisayara sığmayacak kadar büyük olabilir. Hocanın örneği: Walmart 100.000 ürün satar ve milyarlarca sepeti olabilir. Bu kadar büyük veride sık satılan öğeleri bulmak için hem hesaplama miktarı hem bellek ihtiyacı çok büyür. Web'de de durum aynıdır: Milyarlarca kelime ve milyarlarca sayfa vardır. Bu yüzden büyük veri algoritmalarına ihtiyaç vardır.

Hoca planı şöyle açıkladı: Önce sık öğe kümesi bulan özgün algoritmalardan birine, sonra büyük ölçekli veride sık öğe kümesi bulan büyük veri algoritmalarına bakılacak.

5. Birliktelik Kuralları ve Güven

Sık öğe kümesi madenciliğinde nihai amaç birliktelik kuralları (association rules) bulmaktır. Önce birlikte sık satılan öğeler bulunur, hemen ardından bunlardan kurallar üretilir.

Bir birliktelik kuralı şu biçimdedir:

$$\{i_1, i_2, \dots, i_k\} \to j$$

Anlamı şudur: Sol taraftaki öğelerin hepsini içeren bir sepet büyük olasılıkla j'yi de içerir. Kural sık öğe kümelerinden üretilir. Yani bu kural varsa, i_1'den i_k'ye kadar olan öğeler ile j birlikte sık öğe kümesi sonucunda yer alır.

5.1 İki Eşik: Destek ve Güven

Şimdiye kadar destek eşiğini öğrendik. Bir öğe kümesini içeren sepet sayısı destek eşiğinin üstündeyse o küme sıktır. Oyuncak örnekte bunun için sayının 3'ün üstünde olması gerekiyordu.

İkinci eşik güven eşiğidir (confidence threshold). Bu eşik kuralın kendisiyle ilgilidir. Destek eşiğiyle sık kümeler bulunduktan sonra kural kurulur ve kuralın geçerli olması için güvenine bakılır.

Bir kuralın güveni şudur: Veri kümesinde kuralın sol tarafını içeren sepetler arasında, sağ tarafı da içerenlerin oranı. Başka bir deyişle, $i_1, \dots, i_k$ verildiğinde j'nin olma olasılığıdır:

$$conf(I \to j) = \frac{support(I \cup \{j\})}{support(I)} = P(j \mid i_1, \dots, i_k)$$

Bir kuralın geçerli olması için güveninin belirli bir eşiğin üstünde olması gerekir. Bu yüzden işlem verisinden birliktelik kuralı çıkarılırken her zaman iki eşik verilir:

- **Destek eşiği** s**:** Sık öğe kümelerini bulmak için.
- **Güven eşiği** c**:** Birliktelik kuralını belirlemek için.

5.2 Güven Örneği

Aynı oyuncak örnekte güven eşiği %50 olsun. Güveni %50'nin üstünde olan her kural geçerli sayılır. Kural: $\{m, b\} \to c$.

- **Verilen:** Güven eşiği %50.
- **Adım 1:** $\{m, b\}$ içeren sepetler sayılır: 1, 2, 3, 4. Yani 4 sepet.
- **Adım 2:** Bu 4 sepetten c'yi de içerenler sayılır: 1, 2. Yani 2 sepet.
- **Sonuç:** $conf = 2/4 = \%50$. Kural eşiği sağlar ve geçerlidir.

5.3 Birliktelik Kuralı Madenciliği

Birliktelik kuralı madenciliğinde aranan şey, desteği destek eşiğinin, güveni güven eşiğinin üstünde olan bütün kurallardır. Hocanın aktardığı tanıma göre bir kuralın desteği, soldaki öğe kümesinin desteğidir.

Bir kuralın desteği ve güveni yüksekse hem sol taraf hem de sol taraf ile sağdaki öğe birlikte sıktır. Bu yüzden iş iki aşamalıdır:

1. Destek eşiğiyle sık öğe kümeleri bulunur.
2. Her sık kümeden sırayla bir öğe sağ tarafa, kalanlar sol tarafa konur. Kuralın güveni güven eşiğiyle karşılaştırılır. Sonra başka bir öğe sağa alınır ve aynı kontrol yapılır.

   Hocanın vurgusu: Birliktelik kuralı madenciliğinin zor kısmı sık öğe kümelerini bulmaktır. Sık kümeler bulunduktan sonra kural çıkarmak kolaydır.

6. Sık Kümelerden Kural Çıkarma Algoritması

Hoca, sık kümelerin elimizde olduğunu varsayarak kuralları çıkaran algoritmayı anlattı:

1. Desteği en az $c \cdot s$ olan bütün öğe kümelerini bul. Burada s destek eşiği, c güven değeridir.
2. Desteği en az s olan bütün öğe kümelerini bul.
3. Desteği en az $c \cdot s$ olan bir $\{i_1, \ldots, i_k, j\}$ kümesi için, her seferinde bir öğe çıkar ve kalan alt kümenin desteğinin en az s olup olmadığına bak. Bunu 2. adımın sonucundan okuyabilirsin: Kalan öğeler 2. adımın sonucunda varsa, desteği en az s'dir.
4. Örneğin j çıkarıldığında kalan kümenin desteği en az s ise bu kabul edilebilir bir kuraldır. Sol tarafın desteği s, sol ve sağ birlikte $c \cdot s$ desteğe sahiptir.
5. Sol tarafın desteği S_1, bütün kümenin desteği S_2 ise kuralın güveni S_2/S_1'dir.

İlk iki adım, verilen destek eşiklerine göre sık öğe kümesi bulma adımlarıdır. Bunlar yapılabilirse, bir süpermarket ya da işlem verisindeki bütün birliktelik kuralları otomatik olarak bulunur.

Zor olan kısım yine aynıdır: Veri çok büyükken verilen destek eşiğine göre sık öğe kümelerini nasıl bulacağız?

7. Verinin Saklanışı ve Maliyet Ölçüsü

7.1 Düz Dosyalar

Pazar sepeti verisi genelde düz dosyalarda (flat files) tutulur ve bu dosyalar diskte saklanır. Veri sepet sepet saklanır: Önce birinci sepetteki bütün öğeler, sonra ikinci sepettekiler gelir ve böyle devam eder. Sepetler arasında genelde bir tanımlayıcı ya da bayrak bulunur. Çoğu veri kümesinde sepetleri ayırmak için -1 kullanılır.

7.2 Maliyet: Disk Geçişi Sayısı

Bu algoritmaların zaman karmaşıklığına bakarken CPU'daki işlem süresine bakılmaz. Bakılan şey disk giriş-çıkışı sayısıdır, yani algoritmanın veriyi okumak için fiziksel diske kaç kez gittiğidir. Algoritma veriyi okumak için her diske ya da veritabanına gittiğinde toplam süre, CPU süresine göre muazzam artar. Bu yüzden geçiş (pass) sayısı, yani diske erişim sayısı, süre ihtiyacını belirlemede çok önemlidir.

> **Hocanın vurgusu:** Bu algoritmalarda maliyet CPU süresiyle değil, veri üzerinden yapılan geçiş sayısıyla ölçülür.

7.3 Ana Bellek Kritik Kaynaktır

Diğer konu ana bellektir. Sayılması gereken öğe sayısı büyükse ana bellek yetmeyebilir. Bu yüzden belleği verimli kullanan çözümler gerekir. Hocaya göre, örneğin verinin yarısını bellekte saymak gibi bir şey yapamayız. Sayım yapılırken sayılacak her şey fiziksel bellekte sayılmalıdır.

7.4 Neden Yalnız Çiftler?

Sık öğe kümesi bulurken sık çiftler (pairs), sık üçlüler, sık dörtlüler vb. bulunabilir. Fakat en yaygın çalışılan problem sık çiftleri bulmaktır. Büyük ölçekli veride sık çiftler bulunabilirse bu değerli bir bilgidir, çünkü gerçek hayattaki uygulamalarda çoğunlukla sık çiftler kullanılır. Bu yüzden derste sık çiftlere odaklanıldı. Üçlüler, dörtlüler vb. kapsam dışı bırakıldı.

8. Saf (Naive) Algoritma

Aradan sonra hoca saf algoritmaya geçti. Hiç bellek kısıtı olmayan bir bilgisayarımız olduğunu varsayalım. O zaman n öğeden çıkabilecek bütün olası çiftler sayılabilir.

n öğeden kaç olası çift çıkar? İki öğenin kombinasyonu:

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

Algoritma şöyle çalışır:

1. Bir karma tablosu (hash table) ya da tablo kurulur. Bir sütunda çift, diğer sütunda sayısı tutulur.
2. Veri kümesi üzerinden bir geçiş yapılır. Birinci sepetten son sepete kadar bütün veri bir kez okunur ve tablodaki sayılar güncellenir.
3. Geçiş bitince veri kümesindeki bütün çiftlerin sayıları elde edilir. Sayısı destek eşiğinin üstünde olan çiftler sık çiftlerdir.

Fakat bu uygulanabilir bir yaklaşım değildir. Böyle bir çözümün bellek ihtiyacı bugün mevcut olan her fiziksel belleği aşar. Hoca bellek ihtiyacını n^2 çarpı tamsayı boyutu (4 bayt) olarak verdi. Walmart ya da web sayfası intihali örneklerindeki gibi çok sayıda öğe için bu ihtiyaç devasa olur. Örneğin 100.000 öğeli bir problemde bu kadar çifti sayabilecek bir fiziksel bellek yoktur. Bu yüzden büyük veri çözümleri gerekir.

> **Ders sonrası not:** Hoca bellek ihtiyacını $n^2 \times 4$ bayt olarak verdi. Çiftler sırasız olduğu için sayaç sayısı $\frac{n(n-1)}{2}$'dir; doğrusu yaklaşık $2n^2$ bayttır. $n = 100.000$ için $\binom{n}{2} = 4.999.950.000$ çift ve yaklaşık 20 GB eder ($n^2 \times 4$ ile 40 GB çıkardı).

9. Çift Sayaçlarını Bellekte Tutmak: Üçgen Matris ve Üçlü Tablo

Hoca, belleğin yettiğini varsayarak çiftleri bellekte verimli tutmanın iki yolunu anlattı.

9.1 Üçgen Matris Yaklaşımı

Üçgen matris (triangular matrix) yaklaşımında iki boyutlu bir dizi kullanılır. Örneğin birinci boyut indisi 1, ikinci boyut indisi 2 olan hücreye $\{1, 2\}$ çiftinin sayısı yazılır. Böylece bütün olası çiftlerin sayıları iki boyutlu dizinin hücrelerine yerleştirilir. Her hücre bir tamsayıdır, yani 4 bayt. Tek bellek ihtiyacı hücre sayısı kadardır.

Aynı veri tek boyutlu bir dizide de tutulabilir. Bunun için iki boyutu tamsayılara eşleyen bir yöntem gerekir. Örneğin sıfırdan başlanır: Bu indis 0, bu indis 1 denir ve son indise kadar

gider. Bir çiftin indisleri i ve j bir denkleme verilince, o çiftin tek boyutlu dizideki konumu bulunur.

> **Ek bilgi (derste anlatılmadı):** Kitaptaki eşleme büyük olasılıkla şudur: $1 \leq i < j \leq n$ için çift $\{i, j\}$, tek boyutlu dizide $k = (i-1)\left(n - \frac{i}{2}\right) + j - i$ konumunda tutulur. Bu formülde k 1'den başlar ve çiftleri $\{1,2\}, \{1,3\}, \ldots, \{1,n\}, \{2,3\}, \ldots$ sırasıyla dizer.

9.2 Üçlü Tablo Yaklaşımı

İkinci yaklaşım tablo yaklaşımıdır (tabular approach). Her çift için üç şey tutulur: $[i, j, sayı]$. Yani çiftin birinci öğesinin indisi, ikinci öğesinin indisi ve sayı değeri. Bu yüzden üçgen matriste her çift için 4 bayt gerekirken burada her çift için 12 bayt gerekir.

Bu, dizi yerine bir karma tablosu olarak da düşünülebilir. Anahtar (i, j) çiftinin kendisi, değer sayısıdır. Bu yaklaşımın avantajı şudur: Bütün olası çiftler için bir tamsayı ayrılmaz, çünkü bazılarının sayısı sıfır olabilir. Yalnız veri kümesinde gerçekten bulunan çiftlerin sayı bilgisi tutulur. Bunun bedeli, her çift için birinci ve ikinci öğe için de birer tamsayı tutmaktır.

9.3 Hangisi Seçilmeli?

- Veri kümesinde bulunan çift sayısı bütün olası çiftlerin üçte birinden azsa, tablo yaklaşımı daha az bellek kullanır.
- Üçte birinden fazlaysa üçgen matris daha uygundur.

Bunun nedeni şu hesaptır (ara adım eklendi): Olası çift sayısı P, gerçekten görülen çift sayısı p olsun. Üçgen matris $4P$, tablo $12p$ bayt kullanır. $12p < 4P$ eşitsizliği $p < P/3$ demektir.

10. Apriori Algoritması

Saf yaklaşım bütün olası çiftleri yazıp veri üzerinden bir geçişte her çifti sayıyordu. Apriori algoritması bundan daha iyidir. Apriori de sık çiftleri bulur, ama iki geçişlidir. Saf algoritma tek geçişlidir. Apriori iki geçiş yaparak ana bellek ihtiyacını sınırlar.

10.1 Monotonluk

Algoritma şu özelliğe dayanır: Bir öğe kümesi sıksa, bütün alt kümeleri de sıktır. Yani bir öğe kümesi en az s kez geçiyorsa, onun her alt kümesi de en az s kez geçer.

Tersi de doğrudur: Bir i öğesi s sepette geçmiyorsa, i'yi içeren hiçbir çift s sepette geçemez. Yani bir öğe sık değilse, o öğeyle kurulan hiçbir çift sık değildir. Hoca bunun aynı şeyin tersten söylenişi olduğunu belirtti.

> **Ders sonrası not:** Hoca monotonluğu algoritmanın "varsayımı" (assumption) diye sundu. Bu bir varsayım değil, her zaman doğru olan bir özelliktir: Bir kümeyi içeren her sepet, onun alt kümesini de içerir.

10.2 Geçiş 1

Birinci geçişte sepetler okunur ve her tekil öğenin kaç kez geçtiği bellekte sayılır. Bu adımda bellek ihtiyacı yalnız öğe sayısıyla orantılıdır. 100.000 öğe varsa ihtiyaç 100.000×4 bayttır (yaklaşık 400 KB), çünkü yalnız her tekil öğe sayılır. En az s kez geçen tekil öğelere sık öğeler denir.

10.3 Geçiş 2

İkinci geçişte algoritma yalnız sık olmaya aday çiftleri sayar. Monotonluktan biliyoruz ki bir çiftin sık olabilmesi için alt kümelerinin sık olması gerekir. Bu yüzden sık tekil öğelerden olası sık çift adayları (candidate pairs) kurulur ve yalnız bu adaylar sayılır.

Hoca süreci bellek şemasıyla anlattı:

1. Birinci geçişte fiziksel belleğin bir kısmı ayrılır ve bir tablo kurulur. Veri üzerinden geçilirken karşılaşılan tekil öğeler sayılır.
2. Geçiş bitince bütün tekil öğelerin sayıları elde edilir. Destek eşiğinin altında kalanlar silinir, yani o bellek konumları serbest bırakılır. Bellekte yalnız sık öğeler kalır.
3. İkinci geçiş başlamadan önce sık tekil öğelerin kombinasyonları alınır. Böylece sık olmaya aday bütün çiftler bulunur.
4. İkinci geçiş yapılır. Bu kez yalnız aday çiftler sayılır.
5. Geçiş bitince desteği en az s olmayan çiftler elenir. Kalanlar sık çiftlerdir.

10.4 Üçgen Matrisle Apriori

İkinci geçişte çift sayaçları yine iki yaklaşımla tutulabilir: üçgen matris ya da tablo. Üçgen matris kullanılırsa indislere dikkat etmek gerekir. Birinci geçişte bütün öğeler tablodadır ve 1'den n'ye kadar numaralanır. İkinci geçişin başında ise destek eşiğinin altında kalan öğeler bellekten çıkarılır. Bu yüzden kalan öğeleri izlemek için yeni bir yol gerekir.

Bunun için bir eşleme tablosu kurulur. Yeni bir indis oluşturulur ve bu yeni indis eski indise bağlanır. Böylece bir öğenin özgün öğe tablosunda nerede olduğu bulunabilir. Aday çiftler bu yeni indislerle kurulur. Bu, üçgen matris kullanıldığında tutulması gereken ek bir tablodur.

11. PCY Algoritması

Apriori'nin bir zayıflığı şudur: Birinci geçişte belleğin geri kalanını kullanmaz. Tekil öğeleri saymak için belleğin yalnız küçük bir kısmını kullanır, büyük kısmı boşta kalır.

PCY algoritması birinci geçişte belleğin geri kalanını da kullanır. Adını algoritmayı ilk kez sunan yazarlardan alır: Park, Chen ve Yu. Apriori'yi iyileştirir, belleği daha da az kullanır ve daha iyi sonuç verir.

11.1 Geçiş 1

Belleğin boş kalan kısmına bir karma tablosu kurulur. Bir karma fonksiyonu, çiftin bilgisini (birinci indis ve ikinci indis) anahtar olarak alır ve onu karma tablosunun girdilerinden birine (kovaya, bucket) eşler.

Veri birinci sepetten son sepete kadar okunurken:

- Tekil öğeler sayılır.
- Sepetten çıkarılabilecek bütün olası çiftler de üretilir. Örneğin birinci sepette birinci ve ikinci öğe bir çift, birinci ve üçüncü öğe ikinci bir çifttir.
- Her çift karma tablosuna karmalanır ve o girdinin sayacı 1 artırılır.

Geçiş bitince hem öğelerin sayıları hem de karma tablosuna karmalanmış çiftlerin kova sayıları elde edilir. Birinci geçişin alan ihtiyacı öğeler için Apriori ile aynıdır: her öğe için 4 bayt. Fark, belleğin geri kalanının da kullanılmasıdır.

Hoca birinci geçişin sözde kodunu gösterdi:

```
for (her sepet):
    for (sepetteki her öğe):
        öğenin sayacına 1 ekle
    for (sepetteki her çift):
        çifti bir kovaya karmala
        o kovanın sayacına 1 ekle
```

Dış döngü veri kümesi üzerindedir; her turda bir sepet incelenir. Birinci iç döngü sepetteki öğelerin sayaçlarını günceller. İkinci iç döngü bütün olası çiftleri (birinci ile ikinci, birinci ile üçüncü, birinci ile dördüncü vb.) üretir, her birini bir kovaya karmalar ve sayacı bir artırır.

11.2 Geçişler Arasında: Bit Eşlem

Birinci geçişten sonra Apriori'de olduğu gibi destek eşiğinin altındaki öğeler atılır, bellekte yalnız sık öğeler kalır.

Kova sayısı destek eşiğinin üstünde olan girdiler aday sayılır. Altında kalanlara karmalanan çiftler sık olamaz. Bunu izlemek için bir bit eşlem (bitmap) kullanılır. Bit eşlemin elemanları 0 ve 1'dir:

- Kova sayısı eşiğin üstündeyse ilgili hücreye 1 yazılır. Bu, o kovaya düşen çiftin sık olmaya aday olduğunu gösterir.
- Altındaysa 0 yazılır. O kovaya düşen çift sık değildir.

Karma tablosu yerine bit eşlem kullanılınca bu bilgi tablosunun boyutu 1/32'ye iner.

> **Ders sonrası not:** Hoca 1/32 oranını "her bayt 8 bittir" diye açıkladı. Asıl neden şudur: Her kova için 4 baytlık (32 bit) tamsayı sayaç yerine 1 bit tutulur, bu yüzden oran $1/32$'dir.

11.3 Temel Fikir

Bir kova sık değilse, o kovaya karmalanan hiçbir çift sık olamaz. Bir çift sık bir kovaya karmalanıyorsa sık olma olasılığı vardır, ama kesin değildir. Aynı kovaya başka çiftler de düşmüş olabilir. Bu yüzden ikinci geçişte sayılır.

11.4 Geçiş 2

İkinci geçişten önce aday çiftler yine Apriori'deki gibi sık tekil öğelerden kurulur. Fakat bu kez bit eşleme de bakılır. Bir $\{i, j\}$ çifti ancak şu iki koşulu sağlarsa aday olur:

- i ve j sık öğelerdir.
- $\{i, j\}$ çifti bit eşlemde 1 olan bir kovaya karmalanır.

Değer 1 ise çift aday listesine alınır, değilse alınmaz. İkinci geçişte yalnız bu adaylar sayılır. Apriori sık öğelerden kurulan her çifti sayıyordu. PCY yalnız bit eşlemin de onayladığı çiftleri saydığı için bellek tasarrufu sağlar ve yalnız sık olma olasılığı yüksek adayları sayar.

İkinci geçişte aday çiftleri saymak için yine tablo ya da üçgen matris yaklaşımı kullanılabilir.

> **Ders sonrası not:** Hoca burada "aday çiftler olası çiftlerin üçte birinden fazlaysa tablo yaklaşımı mantıklı, değilse üçgen matris kullanılabilir" dedi. Bu, 9.3'teki kendi açıklamasının tersidir. Doğrusu: Sayılacak çiftler üçte birden azsa tablo, fazlaysa üçgen matris daha az bellek kullanır.
>
> **Ek bilgi (derste anlatılmadı):** Kitaba göre büyük olasılıkla PCY'nin ikinci geçişinde üçgen matris pratikte kullanılamaz. Aday çiftler indis düzeninde dağınık olduğu için sıkıştırılmış bir üçgen dizi kurulamaz, üçlü tablo gerekir. Bu yüzden PCY'nin Apriori'den kazançlı çıkması için olası çiftlerin en az üçte ikisini elemesi gerekir.

12. Multistage Algoritması

Hoca kısa bir özet yaptı: Saf yaklaşım uygulanamaz. Apriori iki geçişle fiziksel bellek ihtiyacını azaltır. PCY ise "Apriori iyi, ama daha iyisini yapabilirim" der ve birinci geçişte belleğin geri kalanını kullanarak ikinci geçişteki aday çift sayısını daha da azaltır.

Multistage (çok aşamalı) algoritma PCY'den daha iyi bir yaklaşım sunar. İşi üç geçişte yapar ve böylece belleği çok daha verimli kullanır.

- **Geçiş 1:** Öğeler sayılır. PCY'de olduğu gibi çiftler birinci karma tablosuna karmalanır.
- **Geçiş 2:** Başında bellekte sık öğeler ve birinci karma tablosundan elde edilen birinci bit eşlem vardır. Bu kez ikinci bir karma tablosu kurulur. İkinci tablonun karma fonksiyonu birincisinden farklıdır. Veri yeniden okunur, ama yalnız sık öğelerden oluşan ve birinci bit eşlemde 1'e düşen çiftler ikinci tabloya karmalanır.
- **Geçiş 3:** Başında bellekte sık öğeler, birinci bit eşlem ve ikinci bit eşlem vardır. Aday çiftler sık öğelerden kurulur. Her çift önce birinci, sonra ikinci bit eşleme karşı kontrol edilir. İkisi de "bu çift büyük olasılıkla sık" diyorsa çift aday listesine girer. Üçüncü geçişte yalnız bu iki kontrolden geçen adaylar sayılır.

Multistage, fiziksel bellek ihtiyacını daha da azaltır ve aday çiftleri saymak için gereken hesaplama miktarını da düşürür. Hoca bunu PCY'ye göre daha iyi bellek sınırları sağlayan ve aynı işi yapan bir algoritma olarak özetledi.

13. Multihash Algoritması

Multihash (çok karmalı) algoritma multistage'den farklıdır. Hoca, bu algoritmanın multistage'in sağladığı faydayı PCY gibi iki geçişte elde etmeye çalıştığını anlattı.

- **Geçiş 1:** Belleğin kalan kısmının yarısı birinci karma tablosuna, diğer yarısı ikinci karma tablosuna ayrılır. İki tablonun karma fonksiyonları farklıdır.
- **Geçişler arasında:** Birinci karma tablosu için bir bit eşlem, ikinci karma tablosu için başka bir bit eşlem oluşturulur.
- **Geçiş 2:** Aday çiftler sık öğelerden kurulur. Her aday birinci ve ikinci bit eşleme karşı kontrol edilir. İkisi de onaylıyorsa listeye alınır. İkinci geçişte yalnız bu adaylar sayılır. Bu da yapılması gereken sayım miktarını azaltır.

Hoca, multihash'in bir bakıma hem PCY'yi hem de multistage'i içerdiğini, aynı işi yaptığını ve saf yaklaşım ile Apriori'ye göre daha az bellek gerektirdiğini söyledi.

> **Ders sonrası not:** Hoca multihash'i saf yaklaşım ve Apriori ile karşılaştırdı. Kitaptaki asıl karşılaştırma büyük olasılıkla PCY iledir: Her karma tablosu yarı boyutta olduğu için kova sayısı yarıya iner ve kova başına ortalama sayı artar. Multihash, ortalama kova sayısı destek eşiğinden çok küçükse fayda sağlar; aksi halde kovaların çoğu sık olur ve eleme zayıflar.

14. Algoritmaların Karşılaştırması

Hoca dersi şu sıralamayla bağladı:

Algoritma	Geçiş sayısı	Temel fikir
Saf	1	Bütün olası çiftleri say. Bellek yetmez, uygulanamaz.
Apriori	2	Önce sık öğeleri bul, sonra yalnız sık öğelerden kurulan çiftleri say. Saf yaklaşımın iyileştirmesi.
PCY	2	Birinci geçişte boş belleğe çift kovaları koy; bit eşlemle aday sayısını azalt.
Multistage	3	İkinci geçişte ikinci bir karma tablosu; aday iki bit eşlemden geçmeli.
Multihash	2	Birinci geçişte iki karma tablosu; aday iki bit eşlemden geçmeli.

PCY, multistage ve multihash Apriori'nin iyileştirmeleridir. Apriori de saf yaklaşımın iyileştirmesidir.

> **Hocanın vurgusu:** PCY, multistage ve multihash, büyük ölçekli veri kümelerinde sık öğe kümelerini bulmak için bugün kullanılan algoritmalardır. Buldukları sık öğe kümeleri, birliktelik kuralları üretmek için girdi olarak kullanılır.

Örnek Sorular

Bu derste öğrenci sorusu olmadı. Aşağıdaki sorular hocanın derste sınıfa sorduğu ya da slayttaki örnek üzerinden çözdüğü sorulardır.

Soru 1 (derste sözlü, 00:27:44): 5 öğe (m, c, b, j, p) ve 8 sepetten oluşan veri kümesinde destek eşiği $s = 3$ ise sık öğe kümeleri hangileridir?

Çözüm (derste): Tekli kümelerde m, c, b ve j en az 3 sepette geçtiği için sıktır. İkili kümelerde $\{m, b\}$ (4 sepet) ve $\{b, c\}$ (4 sepet) sıktır. Altyazıda "J and J" diye geçen üçüncü bir çift de sıktır; büyük olasılıkla $\{c, j\}$ (3 sepet).

Soru 2 (derste sözlü, 00:46:22): Güven eşiği %50 ise $\{m, b\} \to c$ kuralı geçerli midir?

Çözüm (derste): $\{m, b\}$ içeren sepet sayısı 4'tür. Bunlardan c'yi de içeren 2 sepet vardır. Güven $2/4 = \%50$'dir. Kural eşiği sağlar.

Soru 3 (derste sözlü, 01:12:52): n öğeden kaç olası çift kurulabilir?

Çözüm (derste): $\binom{n}{2} = \frac{n(n-1)}{2}$. Bu, saf algoritmanın tutması gereken sayaç sayısıdır.

Dersin Toparlaması

Bu derste kitabın 6. bölümü tamamlandı. Pazar sepeti modelinde öğeler ve sepetler tanımlandı. Destek eşiğiyle sık öğe kümeleri, güven eşiğiyle birliktelik kuralları belirlendi. Sık kümelerden kural çıkarmanın kolay, asıl zor işin büyük veride sık kümeleri bulmak olduğu vurgulandı. Bu algoritmalarda maliyetin disk geçişi sayısıyla ölçüldüğü ve ana belleğin kritik kaynak olduğu açıklandı.

Ardından sık çiftleri bulan algoritmalar sırayla incelendi. Saf algoritma tek geçişte bütün çiftleri sayar ama belleğe sığmaz. Çift sayaçları üçgen matris (çift başına 4 bayt) ya da üçlü tablo (görülen çift başına 12 bayt) ile tutulabilir. Apriori, monotonluk özelliğiyle iki geçişte yalnız sık öğelerden kurulan çiftleri sayar. PCY, birinci geçişte boş belleği bir karma tablosuyla kullanır ve bit eşlemle aday sayısını azaltır. Multistage üçüncü bir geçiş ve ikinci bir karma tablosu, multihash ise aynı geçişte iki karma tablosu kullanarak adayları daha da eler.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.